

Universidade Federal de Minas Gerais

Escola de Engenharia

Laboratório de Sistemas I — PLSI

Prof. Diogo Batista de Oliveira

PillHub

Sistema Automatizado de Dispensação de Medicamentos

Relatório Final

Felippe Veloso Marinho

Igor Braga de Lima

José Santos Queiroz

Luiz Daniel de Lima Pião

Raissa Gonçalves Diniz

Belo Horizonte, junho de 2026

Sumário

Resumo	3
1 Introdução	4
1.1 Contexto	4
1.2 Problema	4
1.3 Objetivos	4
2 Descrição do Sistema	5
2.1 Visão Geral	5
2.2 Público-Alvo e Contexto de Uso	5
2.3 Funcionalidades Principais	5
2.4 Componentes de Hardware	6
2.5 Casos de Uso	6
3 Arquitetura do Sistema	7
3.1 Visão em Camadas	7
3.2 Descrição dos Módulos	7
3.3 Fluxo de Comunicação Típico	8
4 Implementação	9
4.1 Firmware (ESP32-C3)	9
4.2 Backend (FastAPI)	10
4.3 Frontend (React)	11
4.4 Banco de Dados	11
4.5 Orquestração com Docker Compose	11
5 Validação de Requisitos	12
5.1 Metodologia	12
5.2 Resumo dos Resultados	12
5.3 Detalhamento dos Casos de Teste	13
5.3.1 TC01 — Modo de Operação Normal (Alertas Sonoros e Visuais) . .	13
5.3.2 TC02 — Modo de Operação Normal (Alertas Não-Sonoros / Silen- cioso)	14
5.3.3 TC03 — Modo de Reabastecimento	15
5.3.4 TC04 — Dispensação de Comprimido Único	15
5.3.5 TC05 — Dispensação no Horário Agendado	16
5.3.6 TC06 — Confirmação de Retirada	17
5.3.7 TC07 — Integridade Estrutural do Slot de Retirada	17
5.3.8 TC08 — Alerta de Código Ativado em Horário Configurado	18

5.3.9	TC09 — Notificação ao Cuidador/Médico Associado	18
5.3.10	TC10 — Conectividade: Controlador Alcança Servidor	19
5.3.11	TC11 — Capacidade Mínima de 21 Slots	19
5.3.12	TC12 — Resistência a Quedas	20
5.3.13	TC13 — Peso do Dispositivo Inferior a 1 kg	20
5.3.14	TC14 — Esforço de Abertura da Gaveta	21
6	Análise Ética	22
6.1	Impactos Positivos	22
6.2	Riscos e Mitigações	22
7	Histórico do Ciclo de Vida	23
8	Avaliação do Modelo Cascata	24
8.1	O que Funcionou	24
8.2	Dificuldades	24
8.3	Adequação do Modelo	24
9	Entregas Realizadas e Não Realizadas	25
10	Conclusão e Trabalhos Futuros	28
10.1	Conclusão	28
10.2	Trabalhos Futuros	28

Resumo

O Dispenser Inteligente é um sistema embarcado de dispensação automática de medicamentos desenvolvido para promover a autonomia e a segurança de pacientes atendidos por um centro de reabilitação. O sistema é composto por um dispositivo físico baseado no microcontrolador ESP32-C3, um servidor de backend em FastAPI (Python) com banco de dados PostgreSQL, e um painel de controle web em React. O dispenser realiza a liberação de comprimidos em horários programados, emite alertas multimodais (sonoro, visual e vibratório) e registra a confirmação do paciente. Este relatório descreve a arquitetura do sistema, as decisões de implementação, a validação dos requisitos por meio de casos de teste e uma análise crítica do processo de desenvolvimento segundo o modelo cascata.

1 Introdução

1.1 Contexto

O Centro de Reabilitação objeto deste projeto (Centro de Reabilitação Rames Ossen Ali, em Igarapé) atende pacientes encaminhados por unidades básicas de saúde para serviços de fisioterapia, terapia ocupacional e fonoaudiologia. O perfil dos pacientes é heterogêneo: incluem pessoas com Transtorno do Espectro Autista (TEA), Transtorno do Déficit de Atenção e Hiperatividade (TDAH), deficiências físicas e sensoriais (visuais e auditivas), membros amputados, e sequelas neurológicas como acidente vascular cerebral (AVC) e paralisia cerebral, abrangendo todas as faixas etárias, de bebês a idosos.

O centro não administra medicamentos em suas instalações. Os pacientes e seus cuidadores são responsáveis pelo regime medicamentoso em casa, o que cria desafios significativos: esquecimento de doses, erros de horário, dificuldade de manipulação de embalagens convencionais e ausência de feedback ao cuidador em caso de não adesão.

1.2 Problema

O principal problema identificado é a **falta de adesão ao regime medicamentoso domiciliar** por parte de pacientes com comprometimentos cognitivos, motores ou sensoriais. Para pacientes com TEA e TDAH, a rigidez de rotina e a dificuldade de lidar com alertas invasivos tornam o problema ainda mais crítico. A não adesão à medicação é uma das principais causas de internação evitável e piora de quadros clínicos crônicos.

1.3 Objetivos

Objetivo geral: Desenvolver um sistema integrado de dispensação automática de medicamentos que promova a autonomia, a segurança e a qualidade de vida de pacientes com necessidades especiais.

Objetivos específicos:

- Implementar um mecanismo físico de dispensação controlada por software;
- Criar um sistema de alertas multimodal acessível (sonoro, visual e vibratório) com modo silencioso;
- Desenvolver uma interface web de configuração de horários e monitoramento;
- Garantir notificação ao cuidador em caso de não confirmação pelo paciente;
- Registrar histórico de adesão para acompanhamento clínico.

2 Descrição do Sistema

2.1 Visão Geral

O Dispenser Inteligente é um dispositivo de mesa composto por uma roleta com 21 compartimentos individuais, cada um destinado a armazenar a medicação de uma dispensação específica (três dispensações diárias por sete dias). O dispositivo é abastecido semanalmente pelo cuidador e opera de forma autônoma, liberando o compartimento correto nos horários pré-configurados e emitindo alertas até que o paciente confirme a retirada.

O sistema é dividido em quatro camadas funcionais:

1. **Hardware/Firmware:** o dispositivo físico com o microcontrolador, motor, sensores e atuadores;
2. **Backend:** servidor de API responsável pela lógica de negócio, agendamentos e proxy para o hardware;
3. **Frontend:** painel web para configuração, monitoramento e histórico;
4. **Banco de dados:** persistência de pacientes, medicamentos, agendas e registros de dispensação.

2.2 Público-Alvo e Contexto de Uso

O sistema é destinado a uso **exclusivamente residencial**. O paciente interage com o dispositivo físico para confirmar a retirada do medicamento. O cuidador (familiar, enfermeiro domiciliar ou terapeuta) configura o sistema pelo painel web e recebe notificações caso o paciente não responda dentro do prazo definido.

A diversidade do público-alvo impõe requisitos de acessibilidade rígidos: alertas não agressivos para pacientes com TEA, LEDs de alta luminosidade com simbologia de período do dia (sol para manhã, sol com nuvem para tarde, lua para noite), botão físico de confirmação grande e de fácil acionamento, e interface visual com baixa carga cognitiva.

2.3 Funcionalidades Principais

- **Dispensação automática:** liberação do slot correto no horário programado, sem manipulação manual pelo paciente;
- **Rastreamento de confirmação:** o dispositivo expõe o flag `awaiting_confirm` no GET `/status`, sinalizando ao backend quando uma dispensação está pendente de confirmação. O backend pode — e deve — verificar esse estado antes de acionar uma nova dispensação para evitar doses duplicadas;

- **Alertas multimodais simultâneos:**
 - Sonoro: buzzer (modo normal);
 - Visual: LED vermelho para manhã (GPIO 7), LED amarelo para tarde (GPIO 1), LED verde para noite (GPIO 0);
 - Vibratório: micro motor de vibração em modo silencioso;
- **Persistência dos alertas** até confirmação do paciente;
- **Confirmação pelo paciente:** botão físico (GPIO 3) ou confirmação remota via API;
- **Notificação ao cuidador:** caso o paciente não confirme em tempo determinado;
- **Registro de histórico:** cada dispensação e confirmação é registrada para acompanhamento de adesão;
- **Modo reabastecimento:** permite ao cuidador acessar os compartimentos sem acionar a lógica de dispensação.

2.4 Componentes de Hardware

Tabela 1: Componentes de hardware do Dispenser

Componente	Especificação	Peça
Microcontrolador	WiFi, 3–5V, programável	ESP32-C3 SuperMini
Motor de passo	Girar 1 slot por vez, 5V	Micro Servomotor SG90
Alerta sonoro	Mínimo 60 dB, 5V	Buzzer Ativo 5V
Alerta vibratório	Silencioso, estrutura segura	Micro Motor 3V
Alerta visual	Alta luminosidade, 3 períodos	3× LED 5mm
Botão confirmação	Tátil, grande, GPIO 3	Push Button 12×12mm
Alimentação	127V, até 1A	Carregador USB padrão

2.5 Casos de Uso

Os casos de uso do sistema, levantados durante a fase de modelagem, são:

1. Configurar horários de dispensação via painel web;
2. Receber alerta de dispensação (sonoro, visual ou vibratório);
3. Confirmar tomada de medicamento (botão físico ou API);

4. Reabastecer o dispenser (modo reabastecimento);
5. Receber notificação de não adesão (cuidador);
6. Registrar dose no histórico;
7. Visualizar histórico de adesão.

3 Arquitetura do Sistema

3.1 Visão em Camadas

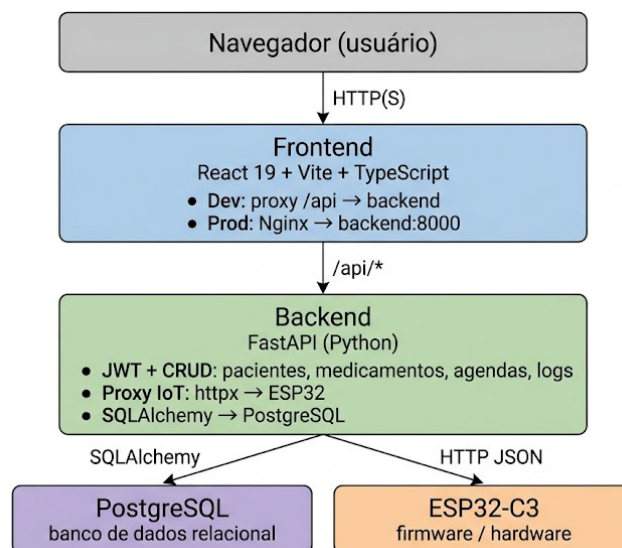


Figura 1: Arquitetura em camadas do Dispenser

3.2 Descrição dos Módulos

Firmware (ESP32-C3): servidor HTTP assíncrono rodando diretamente no microcontrolador. Expõe uma API REST que o backend consome para acionar o hardware e consultar o estado do dispositivo. Opera em rede WiFi local.

Backend (FastAPI): servidor Python que atua como intermediário entre o frontend e o hardware. Gerencia autenticação JWT, persiste dados de pacientes, medicamentos, agendamentos e logs, e faz proxy de requisições IoT para o firmware via cliente HTTP assíncrono (**httpx**).

Frontend (React): painel web responsivo para configuração de horários, monitoramento em tempo real do estado do dispenser e visualização do histórico de adesão. Comunica-se exclusivamente com o backend via prefixo **/api**.

Banco de Dados (PostgreSQL): armazenamento relacional com schema inicializado via `database/init.sql`. Os modelos são gerenciados pelo SQLAlchemy no backend.

3.3 Fluxo de Comunicação Típico

A comunicação entre firmware e backend é baseada em **heartbeat periódico**: o ESP32 é quem inicia a conexão, eliminando a necessidade de o backend conhecer o IP do dispositivo com antecedência.

1. O cuidador acessa o painel web e configura os horários de dispensação (manhã, tarde e noite).
2. O **scheduler** do backend (tarefa assíncrona em background) verifica a cada 10 segundos quais agendamentos estão no prazo; ao detectar um, enfileira um comando **dispense** na tabela `pending_commands`.
3. O firmware envia `POST /api/heartbeat` ao backend a cada 30 segundos (ou imediatamente após uma confirmação do paciente). O corpo inclui `current_slot`, `awaiting_confirm`, `critical_stock`, RSSI e, se aplicável, o ACK do último comando executado.
4. O backend responde ao heartbeat com o próximo comando pendente (**dispense** ou **calibrate**), marcando-o como entregue. Se não há comando, responde com `"command": null`.
5. Ao receber um comando **dispense**, o firmware avança a roleta (servo SG90), acende o LED do período e dispara o alerta: buzzer em modo normal, ou vibração persistente em pulsos até confirmação no **modo silencioso**.
6. O paciente retira o medicamento e pressiona o botão físico de confirmação; o firmware limpa os alertas e agenda um heartbeat imediato para notificar o backend.
7. O backend recebe o ACK no próximo heartbeat, marca o comando como concluído, atualiza o flag `awaiting_confirm` e persiste o evento no log de dispensações.
8. Se não houver confirmação dentro do prazo, o backend notifica o cuidador por e-mail.

4 Implementação

4.1 Firmware (ESP32-C3)

O firmware foi desenvolvido em C++ utilizando o Arduino IDE com o core ESP32 versão **2.0.17** (versões 3.x são incompatíveis com as bibliotecas assíncronas utilizadas). As bibliotecas principais são `ESPAsyncWebServer` e `AsyncTCP`.

A API HTTP exposta pelo firmware possui os seguintes endpoints:

Tabela 2: Endpoints da API do firmware

Método	Rota	Descrição
GET	<code>/status</code>	Retorna estado completo do dispositivo
POST	<code>/dispense</code>	Avança roleta e dispara alertas
POST	<code>/confirm</code>	Confirma retirada e limpa alertas
POST	<code>/calibrate</code>	Reseta roleta para o slot 0
GET	<code>/</code>	Página HTML de diagnóstico

O endpoint `GET /status` retorna um JSON com os campos: `current_slot`, `total_slots` (fixo em 21), `awaiting_confirm`, `last_confirmed_slot`, `wifi_rssi`, `hardware_id` (endereço MAC) e `uptime_s`.

O endpoint `POST /calibrate` reseta o contador de slots para a posição inicial (slot 0), sendo utilizado pelo cuidador após o reabastecimento semanal. Esse reset é realizado exclusivamente via HTTP — não existe botão físico dedicado para calibração.

O `POST /dispense` aceita um objeto JSON com os campos `period` ("morning", "afternoon" ou "night") e `silent_mode` (booleano) e controla qual alerta é disparado. O parsing do JSON é feito manualmente, sem a biblioteca `ArduinoJson`, por incompatibilidades de versão encontradas durante o desenvolvimento.

Além do servidor HTTP, o firmware possui o módulo **heartbeat_client**, que executa periodicamente o seguinte ciclo: (1) constrói um JSON com o estado atual do dispositivo (`current_slot`, `awaiting_confirm`, `critical_stock`, `RSSI`, `uptime` e eventual `ACK` de comando anterior); (2) envia `POST /api/heartbeat` ao backend; (3) processa o campo `command` da resposta, executando a dispensação ou calibração indicada. Esse mecanismo de *pull* elimina a necessidade de o backend conhecer o IP do firmware. Após a confirmação do paciente, o firmware aciona `requestEarlyHeartbeat()`, disparando um heartbeat imediato em vez de aguardar os 30 segundos regulares.

O nível de estoque crítico é calculado dinamicamente pela função `isCriticalStock()`: retorna `true` quando o slot atual é maior ou igual a `TOTAL_SLOTS - 3` (ou seja, restam apenas 3 compartimentos não utilizados). O valor é reportado no heartbeat para que o backend envie alerta ao cuidador.

Em **modo silencioso**, o motor de vibração não pulsa uma única vez: a função `alertsTick()`, chamada a cada iteração do `loop()`, mantém o motor em ciclo de 400 ms ligado / 250 ms desligado enquanto `awaiting_confirm` for `true`. O alerta cessa automaticamente quando o paciente pressiona o botão de confirmação.

A credencial WiFi e a URL do backend são armazenadas em `secrets.h` (excluído do controle de versão via `.gitignore`); na primeira inicialização, são migradas para a NVS do ESP32 para boots futuros.

4.2 Backend (FastAPI)

O backend é uma aplicação Python com o framework **FastAPI** e servidor ASGI **Uvicorn**. O gerenciamento de pacotes usa **uv**. As rotas são organizadas por domínio:

- `/api/auth` — autenticação (registro, login, perfil) com JWT;
- `/api/iot` — endpoints IoT: `POST /heartbeat` (principal ponto de contato com o firmware), eventos e status;
- `/api/patients` — CRUD de pacientes;
- `/api/medications` — CRUD de medicamentos;
- `/api/schedules` — gerenciamento de agendamentos;
- `/api/logs` — histórico de dispensações e reabastecimentos;
- `/api/dispensers` — gerenciamento de dispositivos, incluindo agendas periódicas e status de hardware;
- `/api/health` — verificação de saúde do serviço.

O backend possui um **scheduler assíncrono** (tarefa de background iniciada junto com a aplicação) que varre os agendamentos ativos a cada 10 segundos. Quando um horário está dentro da janela de disparo (até 120 s após o horário configurado), o scheduler enfileira um comando na tabela `pending_commands` do banco de dados (mode *queue*). A deduplicação impede que o mesmo agendamento seja enfileirado mais de uma vez por período, com janela de 180 s. Ao receber um heartbeat do firmware, o endpoint `POST /api/heartbeat` entrega o comando pendente mais antigo na resposta JSON e processa o ACK enviado pelo ESP do comando anterior.

As notificações por e-mail são enviadas via SMTP nos seguintes eventos: cadastro de novo cuidador (boas-vindas), transição para estado de estoque crítico (`critical_stock`), e falha de dispensação. A integração usa `smtplib` com as credenciais configuradas via variáveis de ambiente.

4.3 Frontend (React)

O frontend é uma *Single Page Application* (SPA) em **React 19** com **Vite** como ferramenta de build e **TypeScript**. O gerenciador de pacotes é exclusivamente o **Bun** (versão $\geq 1.3.13$), que substitui npm/yarn por oferecer melhor desempenho e suporte nativo a TypeScript.

As principais páginas são: autenticação (login), painel de controle (*dashboard*), gestão de pacientes, gestão de dispensers e histórico de adesão (*/historico*). O dashboard foi redesenhado com uma seção de horários em três cartões visuais (Manhã, Tarde e Noite), cada um com botões de ajuste ± 15 min, e com uma roleta colorida que exibe o status de cada slot (dispensado, atual ou futuro). Um modal de ajuda acessível por botão visível orienta o cuidador no uso do painel. A tela de medicamentos do paciente permite vincular cada medicamento a um slot físico do dispensador. O design segue o **Pillar Design System** com estilo glassmorfismo, priorizando acessibilidade visual.

Em desenvolvimento, as requisições ao prefixo */api* são redirecionadas ao backend via proxy configurado no `vite.config.ts`. Em produção, o Nginx realiza o mesmo roteamento. O linting e a formatação utilizam **Biome** (substituto unificado de ESLint e Prettier).

4.4 Banco de Dados

O banco de dados é o **PostgreSQL 16**. O schema inicial é provisionado via `database/init.sql` na primeira inicialização do volume Docker. Os modelos de domínio são definidos em SQLAlchemy e incluem entidades para usuários, pacientes, medicamentos, agendamentos, dispensers, fila de comandos pendentes (`pending_commands`) e logs de dispensação.

4.5 Orquestração com Docker Compose

Tabela 3: Serviços Docker Compose e perfis

Serviço	Tecnologia	Porta (host)	Perfil
db	PostgreSQL 16	5433	sempre ativo
backend	FastAPI/Uvicorn	8000	sempre ativo
frontend	Nginx + build Bun	8080	prod
frontend-dev	Vite + Bun	80	dev

5 Validação de Requisitos

5.1 Metodologia

A validação dos requisitos foi conduzida por meio de **casos de teste** derivados dos requisitos funcionais e físicos levantados nas práticas anteriores e modelados no Visual Paradigm. Cada caso de teste especifica pré-condições, passos de execução, resultado esperado e resultado obtido.

Os possíveis status de cada caso de teste são:

- **Aprovado** — comportamento observado corresponde ao resultado esperado;
- **Parcialmente Testado** — componente de software verificado; comportamento físico não validado;
- **Não Realizado** — teste não executado por ausência de hardware ou funcionalidade não implementada.

5.2 Resumo dos Resultados

ID	Caso de Teste	Status
TC01	Modo de operação normal (alertas sonoros e visuais)	Aprovado
TC02	Modo de operação normal (alertas não-sonoros/silencioso)	Parcialmente Testado
TC03	Modo de reabastecimento	Não Realizado
TC04	Dispensação de comprimido único	Não Realizado
TC05	Dispensação no horário agendado	Aprovado
TC06	Confirmação de retirada	Aprovado
TC07	Integridade estrutural do slot de retirada	Não Realizado
TC08	Alerta de código ativado em horário configurado	Aprovado
TC09	Notificação ao cuidador/médico associado	Parcialmente Testado
TC10	Conectividade: controlador alcança servidor	Aprovado
TC11	Capacidade mínima de 21 slots	Aprovado
TC12	Resistência a quedas	Não Realizado
TC13	Peso do dispositivo inferior a 1 kg	Não Realizado
TC14	Esforço de abertura da gaveta $\leq 0,2$ kgf	Não Realizado

Tabela 4: Resumo dos casos de teste

5.3 Detalhamento dos Casos de Teste

5.3.1 TC01 — Modo de Operação Normal (Alertas Sonoros e Visuais)

Pré-condição	Dispositivo ligado, conectado ao WiFi, slot na posição inicial após calibração.
Passos	1. Enviar POST /dispense com <code>silent_mode: false</code> e <code>period: "morning"</code>. 2. Verificar que o servo avança um slot. 3. Verificar que o LED de manhã (GPIO 7) acende. 4. Verificar que o buzzer emite bipes audíveis.
Resultado esperado	Remédios dispensados; alertas sonoros e visuais ativados simultaneamente.
Resultado obtido	HTTP 200 com <code>success: true</code> . Validação física em 23/05/2026: LED de manhã (GPIO 7) acendeu corretamente, buzzer emitiu 3 bipes. Avanço do servo verificado via <code>current_slot</code> .
Status	Aprovado

5.3.2 TC02 — Modo de Operação Normal (Alertas Não-Sonoros / Silencioso)

Pré-condição	Dispositivo ligado, conectado ao WiFi; agendamento configurado com <code>silent_mode: true</code> .
Passos	1. Configurar agendamento com <code>silent_mode: true</code> via frontend. 2. Aguardar o horário agendado; o scheduler enfileira o comando com <code>silent_mode: true</code>. 3. Verificar que o firmware, ao receber o comando via heartbeat, avança a roleta. 4. Verificar que o motor de vibração entra em ciclo persistente (400 ms ligado / 250 ms desligado). 5. Verificar que o buzzer permanece silencioso. 6. Pressionar o botão de confirmação; verificar que o motor para imediatamente.
Resultado esperado	Remédios dispensados; alerta vibratório persistente ativado sem emissão de som; vibração cessa na confirmação.
Resultado obtido	Campo <code>silent_mode</code> propagado corretamente do frontend ao banco de dados, ao scheduler e ao comando via heartbeat. Função <code>alertsTick()</code> implementada e integrada ao <code>loop()</code> . Vibração detectada fisicamente (intensidade dependente da tensão de alimentação).
Status	Parcialmente Testado

5.3.3 TC03 — Modo de Reabastecimento

Pré-condição	Dispositivo em modo de operação normal.
Passos	1. Ativar o modo de reabastecimento via interface. 2. Tentar abrir os compartimentos manualmente. 3. Verificar que os slots não se movem durante a manipulação. 4. Sair do modo de reabastecimento e configurar modo desejado.
Resultado esperado	Compartimentos acessíveis sem acionamento de dispensação; ao sair do modo, sistema opera normalmente.
Resultado obtido	Não realizado. A interface de alternância para o modo de reabastecimento não foi implementada na versão atual.
Status	Não Realizado

5.3.4 TC04 — Dispensação de Comprimido Único

Pré-condição	Todos os slots abastecidos com pelo menos um comprimido.
Passos	1. Enviar POST /dispense para acionar uma dispensação. 2. Verificar fisicamente que apenas um comprimido é dispensado. 3. Verificar que o comprimido correto chega ao slot de retirada.
Resultado esperado	Apenas um comprimido dispensado por interação; compartimento correto aberto.
Resultado obtido	Não realizado. O firmware controla o servo corretamente por software, mas a verificação do comprimido físico exige o dispositivo montado.
Status	Não Realizado

5.3.5 TC05 — Dispensação no Horário Agendado

Pré-condição	Backend em execução; ESP32 conectado ao WiFi e enviando heartbeats; agendamento configurado para horário X via frontend.
Passos	1. Configurar agendamento de dispensação para horário X via frontend. 2. Aguardar o horário X. 3. Verificar que o scheduler enfileira o comando na tabela <code>pending_commands</code>. 4. Verificar que o firmware recebe o comando no próximo heartbeat e avança a roleta. 5. Verificar que os alertas são ativados com o período e modo corretos.
Resultado esperado	A medicação do slot esperado é dispensada no horário X com os alertas configurados.
Resultado obtido	Validado em integração completa em 08/06/2026: o scheduler detectou o horário dentro da janela, enfileirou o comando <code>dispense</code> e o firmware executou a dispensação no heartbeat seguinte (latência máxima de 30 s). Servo, LED e buzzer ativados corretamente. Mecanismo de deduplicação (janela de 180 s) funcionou.
Status	Aprovado

5.3.6 TC06 — Confirmação de Retirada

Pré-condição	Uma dispensação foi executada; <code>awaiting_confirm</code> é <code>true</code> .
Passos	1. Enviar <code>POST /confirm</code> ao firmware. 2. Verificar que <code>awaiting_confirm</code> passa a <code>false</code> em <code>GET /status</code>. 3. Verificar que <code>confirmed_slot</code> corresponde ao slot dispensado.
Resultado esperado	Confirmação registrada, alertas limpos, estado atualizado.
Resultado obtido	Teste via suíte pytest: <code>POST /confirm</code> retornou HTTP 200; <code>GET /status</code> confirmou <code>awaiting_confirm: false</code> e <code>confirmed_slot</code> correto.
Status	Aprovado

5.3.7 TC07 — Integridade Estrutural do Slot de Retirada

Pré-condição	Dispositivo fisicamente montado.
Passos	1. Abrir manualmente o slot de retirada; verificar que o compartimento se abre e mantém integridade. 2. Durante uma dispensação, verificar que os demais slots permanecem fechados. 3. Simular turbulência; verificar que os slots não abrem inadvertidamente.
Resultado esperado	Apenas o slot ativo abre; demais permanecem fechados sob turbulência.
Resultado obtido	Não realizado. Exige o dispositivo físico montado com a estrutura da roleta.
Status	Não Realizado

5.3.8 TC08 — Alerta de Código Ativado em Horário Configurado

Pré-condição	Firmware em execução; ESP32 conectado ao WiFi; agendamento configurado para horário X.
Passos	1. Agendar alerta para horário X com período e modo de alerta específicos. 2. Aguardar o horário X. 3. Verificar que o alerta é ativado com as configurações corretas (LED do período, buzzer ou vibração conforme <code>silent_mode</code>).
Resultado esperado	O alerta é ativado dentro da janela de até 30 s após o horário X, com período e modo corretos.
Resultado obtido	Validado em integração completa em 08/06/2026: LED do período acendeu, buzzer soou imediatamente após a entrega do comando via heartbeat, com latência de até 30 s conforme esperado pela arquitetura de polling.
Status	Aprovado

5.3.9 TC09 — Notificação ao Cuidador/Médico Associado

Pré-condição	Cuidador com e-mail cadastrado no sistema; dispensação executada ou estoque crítico detectado.
Passos	1. Verificar que o e-mail de notificação chega ao cuidador nos eventos configurados. 2. Verificar que notificação de estoque crítico é enviada quando <code>critical_stock</code> transiciona para <code>true</code>.
Resultado esperado	O cuidador vinculado recebe e-mail nas situações relevantes.
Resultado obtido	Notificações por e-mail implementadas via SMTP para: cadastro de cuidador (boas-vindas), transição para estoque crítico e falha de dispensação. Notificações push (Firebase Cloud Messaging) não foram implementadas.
Status	Parcialmente Testado

5.3.10 TC10 — Conectividade: Controlador Alcança Servidor

Pré-condição	ESP32 conectado ao WiFi; computador de teste na mesma rede.
Passos	1. Executar <code>GET /status</code> no endereço IP do ESP32; verificar resposta HTTP 200. 2. Executar dois <code>GET /status</code> com intervalo de 2 s; verificar que <code>uptime_s</code> aumentou. 3. Verificar que <code>wifi_rssi</code> está entre -100 dBm e 0 dBm.
Resultado esperado	Servidor HTTP do firmware responde corretamente; uptime cresce; RSSI válido.
Resultado obtido	Todos os três testes de conectividade da suíte pytest passaram: resposta HTTP 200, crescimento de uptime e RSSI dentro do intervalo válido.
Status	Aprovado

5.3.11 TC11 — Capacidade Mínima de 21 Slots

Pré-condição	Firmware em execução.
Passos	1. Executar <code>GET /status</code>. 2. Verificar que o campo <code>total_slots</code> é maior ou igual a 21.
Resultado esperado	O dispenser possui ao menos 21 reservatórios (3 dispensações/dia $\times$ 7 dias).
Resultado obtido	<code>GET /status</code> retornou <code>total_slots: 21</code> . Confirmado pelo teste automatizado <code>test_total_slots_e_sempre_21</code> .
Status	Aprovado

5.3.12 TC12 — Resistência a Quedas

Pré-condição	Dispositivo fisicamente montado e abastecido.
Passos	1. Simular queda do dispositivo de altura equivalente ao uso doméstico (bancada). 2. Verificar integridade estrutural e funcional após o impacto.
Resultado esperado	O dispenser não quebra e mantém funcionalidade após queda leve.
Resultado obtido	Não realizado. Exige o dispositivo físico completo.
Status	Não Realizado

5.3.13 TC13 — Peso do Dispositivo Inferior a 1 kg

Pré-condição	Dispositivo fisicamente montado e abastecido com comprimidos.
Passos	1. Pesar o dispositivo em balança. 2. Verificar que o resultado é inferior a 1 kg.
Resultado esperado	Peso < 1 kg para garantir portabilidade, especialmente para idosos.
Resultado obtido	Não realizado. O dispositivo físico completo não estava disponível.
Status	Não Realizado

5.3.14 TC14 — Esforço de Abertura da Gaveta

Pré-condição	Dispositivo fisicamente montado com mecanismo de gaveta.
Passos	1. Medir a força de abertura da gaveta com dinamômetro. 2. Verificar que não excede 0,2 kgf. 3. Repetir para o fechamento.
Resultado esperado	Força de abertura e fechamento $\leq 0,2$ kgf, garantindo acessibilidade a pacientes com força reduzida.
Resultado obtido	Não realizado. Exige o dispositivo físico e um dinamômetro.
Status	Não Realizado

6 Análise Ética

6.1 Impactos Positivos

O Dispenser endereça diretamente a autonomia de pacientes com limitações cognitivas e motoras. Os principais benefícios identificados são:

- **Autonomia e dignidade:** o paciente gerencia sua própria medicação sem auxílio constante;
- **Segurança:** o flag de confirmação pendente e os alertas persistentes reduzem o risco de dose esquecida; o backend pode usar esse estado para evitar dispensações duplicadas;
- **Redução de carga sobre cuidadores:** notificados apenas quando necessário, sem supervisão contínua;
- **Registro clínico objetivo:** o histórico de adesão fornece dados para o acompanhamento terapêutico;
- **Acessibilidade como requisito central:** o design multimodal foi definido com base no perfil heterogêneo dos usuários, não como funcionalidade adicional.

6.2 Riscos e Mitigações

- **Falha de dispensação:** uma falha de hardware pode resultar em dose não administrada. *Mitigação:* alertas redundantes (sonoro + visual + vibratório), registro de falhas em log e calibração física pelo cuidador;
- **Privacidade de dados de saúde:** o sistema armazena dados sensíveis de pacientes vulneráveis. *Mitigação:* autenticação JWT, dados trafegados em rede local, acesso restrito por perfil de usuário;
- **Dependência tecnológica:** falha de energia ou rede interrompe o sistema. *Mitigação prevista:* modo de operação offline com alertas físicos mesmo sem conectividade ao backend (trabalho futuro);
- **Erro de configuração pelo cuidador:** configurações erradas podem comprometer o tratamento. *Mitigação:* interface simplificada com confirmação de agendamentos e validação de dados no backend.

7 Histórico do Ciclo de Vida

O projeto foi desenvolvido segundo o modelo cascata. A Tabela 5 apresenta as entregas realizadas ao longo do semestre.

Tabela 5: Histórico de entregas — modelo cascata

Fase	Entrega	Período	Status
Requisitos	Entrevista com stakeholders e levantamento (Lab 1)	2026/1	Concluído
Requisitos	Especificação estruturada de requisitos (Prática 2)	2026/1	Concluído
Projeto	Seleção de componentes e modelagem no Visual Paradigm (Prática 6)	2026/1	Concluído
Implementação	Firmware: servidor HTTP, servo, alertas	2026/1	Concluído
Implementação	Backend: FastAPI, proxy IoT, CRUD, JWT	2026/1	Concluído
Implementação	Frontend: dashboard React	2026/1	Concluído
Implementação	Banco de dados: schema PostgreSQL	2026/1	Concluído
Implementação	Orquestração Docker Compose	2026/1	Concluído
Implementação	Notificações push (Firebase)	2026/1	Não realizado
Implementação	Descoberta automática do ESP (mDNS)	2026/1	Em andamento
Testes	Testes HTTP automatizados do firmware (pytest)	2026/1	Parcial
Testes	Testes físicos de hardware	2026/1	Não realizado
Entrega Final	Relatório + vídeo + apresentação	2026/1	Em andamento

8 Avaliação do Modelo Cascata

8.1 O que Funcionou

O modelo cascata trouxe benefícios concretos na fase inicial. A obrigatoriedade de documentar requisitos antes de qualquer implementação levou a entrevistas estruturadas com o centro de reabilitação e a uma especificação detalhada do público-alvo. Isso evitou decisões de design inconsistentes — o requisito de modo silencioso, por exemplo, surgiu das entrevistas e não seria evidente sem esse processo.

A separação em fases com entregas definidas também facilitou a divisão de responsabilidades na equipe: firmware, backend, frontend e banco de dados foram desenvolvidos com contratos de interface claros.

8.2 Dificuldades

- **Requisitos evolutivos:** o problema de descoberta dinâmica do ESP na rede surgiu apenas durante a integração, quando firmware e backend já estavam implementados com IP fixo;
- **Integração tardia:** como cada módulo foi desenvolvido isoladamente, problemas de integração (proxy, formato JSON, CORS) apareceram somente ao final, aumentando o custo de correção;
- **Testes físicos dependentes de hardware:** a fase de testes foi limitada pela indisponibilidade do dispositivo físico completo, algo que o modelo cascata não prevê mecanismos para tratar;
- **Dificuldade de retorno:** quando uma decisão técnica se mostrou inadequada (core ESP32 3.x incompatível com as bibliotecas assíncronas), o modelo cascata dificultou a revisão estruturada.

8.3 Adequação do Modelo

Para um projeto de sistema embarcado com componentes de hardware e software tão interdependentes, um modelo iterativo com ciclos curtos de integração seria provavelmente mais eficaz. O modelo cascata funcionou melhor na fase de requisitos do que nas fases de implementação e testes.

9 Entregas Realizadas e Não Realizadas

Requisito / Funcionalidade	Implementado?	Observação
Servidor HTTP no firmware	Sim	Endpoints <code>/status</code> , <code>/dispense</code> , <code>/confirm</code> , <code>/calibrate</code>
Heartbeat periódico (pull de comandos)	Sim	ESP32 envia POST <code>/api/heartbeat</code> a cada 30 s; backend responde com comando pendente
Confirmação imediata pós-botão	Sim	<code>requestEarlyHeartbeat()</code> aciona heartbeat imediato após confirmação do paciente
Controle do servo (roleta 21 slots)	Sim	Wrap-around após slot 21
Alerta sonoro (buzzer)	Sim	Validado fisicamente
Alerta visual (3 LEDs por período)	Sim	Validado fisicamente
Alerta vibratório persistente	Sim	Ciclo 400 ms/250 ms até confirmação; validado parcialmente
Modo silencioso (configurável)	Sim	Checkbox no frontend; propagado até o firmware via heartbeat
Estoque crítico dinâmico	Sim	<code>isCriticalStock()</code> calcula em tempo real; reportado no heartbeat
Botão de confirmação física	Sim	GPIO 3; validado fisicamente
Botões de volume (VOL_UP/DOWN)	Não	Removidos do escopo; hardware atual não possui esses botões

Requisito / Funcionalidade	Implementado?	Observação
Flag <code>awaiting_confirm</code>	Sim	Exposto via <code>/status</code> e <code>heartbeat</code> ; bloqueio de dupla dose implementado no scheduler
Backend FastAPI com scheduler	Sim	Scheduler assíncrono com fila de comandos e deduplicação
Autenticação JWT	Sim	Login e proteção de rotas
CRUD pacientes e medicamentos	Sim	Modelos e endpoints implementados
Agendamentos de dispensação	Sim	Disparo automático validado em integração
Notificações por e-mail	Sim	Boas-vindas, estoque crítico e falha de dispensação
Histórico de adesão (logs)	Sim	Endpoint de logs + página <code>/historico</code> implementados; marcação automática de doses perdidas no scheduler
Frontend React (dashboard)	Sim	Countdown, agendas, telemetria, modo silencioso, gestão de pacientes
Docker Compose (dev/prod)	Sim	Perfis configurados e funcionais
Notificações push (Firebase)	Não	Firebase não integrado
Descoberta automática do ESP (mDNS)	Não	IP configurado via NVS; mDNS previsto como trabalho futuro

Requisito / Funcionalidade	Implementado?	Observação
Modo reabastecimento (UI)	Não	Fluxo de calibração implementado; UI completa não finalizada
Testes físicos do hardware	Não	Hardware parcialmente disponível; TCs físicos não executados integralmente
Operação offline (sem backend)	Não	Previsto como trabalho futuro

Tabela 6: Balanço de entregas realizadas e não realizadas

10 Conclusão e Trabalhos Futuros

10.1 Conclusão

O Dispenser Inteligente atingiu seus objetivos centrais de MVP: foi desenvolvido um sistema integrado capaz de dispensar medicamentos automaticamente nos horários configurados, emitir alertas multimodais (sonoro, visual e vibratório), registrar confirmações do paciente e notificar o cuidador por e-mail em situações críticas. A arquitetura heartbeat-based eliminou a dependência de IP fixo para comunicação, tornando o sistema mais robusto em redes domésticas. O scheduler assíncrono com fila de comandos e deduplicação garantiu a entrega confiável das dispensações sem disparos duplicados. O frontend oferece configuração de horários, modo silencioso, countdown da próxima dispensação e monitoramento de telemetria em tempo real.

As principais limitações da versão atual são a ausência de notificações push ao cuidador (somente e-mail), a necessidade de configuração manual do endereço de backend no firmware, e a não execução dos testes físicos completos de hardware.

10.2 Trabalhos Futuros

- **Descoberta automática do ESP (mDNS):** o firmware anunciaria o dispositivo como `eco-dispenser.local` na rede local, eliminando a configuração manual da URL do backend no firmware;
- **Notificações push:** integração com Firebase Cloud Messaging (ou similar) para alertar cuidadores em tempo real em caso de não adesão, complementando as notificações por e-mail já implementadas;
- **Testes físicos completos:** execução dos casos de teste de hardware (TC04, TC07, TC12, TC13, TC14) com o dispositivo montado;
- **Operação offline:** firmware com alertas físicos mesmo sem conectividade ao backend, garantindo funcionamento básico em queda de rede;
- **Modo reabastecimento:** implementação da interface e do fluxo completo de reabastecimento semanal;
- **Deploy em produção:** configuração de HTTPS e infraestrutura para uso fora da rede local.

Referências

- [1] Espressif Systems. *ESP32-C3 Series Datasheet*. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf. Acesso em: maio 2026.
- [2] me-no-dev. *ESPAsyncWebServer*. Disponível em: <https://github.com/me-no-dev/ESPAsyncWebServer>. Acesso em: maio 2026.
- [3] Santos, R. *Getting Started with ESP32-C3 Super Mini*. Random Nerd Tutorials. Disponível em: <https://randomnerdtutorials.com/getting-started-esp32-c3-super-mini/>. Acesso em: maio 2026.
- [4] Ramírez, S. *FastAPI Documentation*. Disponível em: <https://fastapi.tiangolo.com>. Acesso em: maio 2026.
- [5] Meta Platforms. *React — The library for web and native user interfaces*. Disponível em: <https://react.dev>. Acesso em: maio 2026.
- [6] Oven. *Bun — A fast all-in-one JavaScript runtime*. Disponível em: <https://bun.sh>. Acesso em: maio 2026.
- [7] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: maio 2026.